Network Resource Reservation System

Project details

Item	Details
Course	Computer Networks mini project (UE25CS243A), Semester 3, PES University
Team number	23
Section	B
Team members	Kritika Singh (PES2UG25AM135) , Jiah Bansal, (PES2UG25AM117) , Lisha Chowdary (PES2UG25AM144)
Project topic	Network Resource Reservation System: a TCP reservation service that installs bandwidth policies through an SDN controller in Mininet
GitHub repository	https://github.com/kritikasingh28/network-resource-reservation

Abstract and objectives

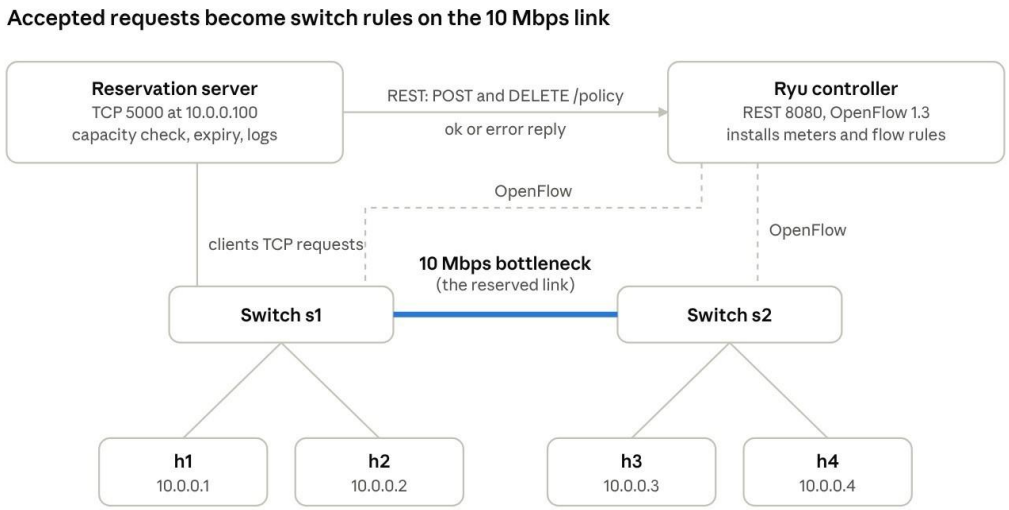
Abstract. This project builds a bandwidth reservation service for a shared network link. A client request Python TCP server for a given bandwidth between two hosts. The server checks the request against the link capacity (10 Mbps) and accepts it only if it fits. For an accepted request, the server calls a Ryu SDN controller, which installs OpenFlow 1.3 ,meters and flow rules on the switches in a Mininet network. Reservations end when the client releases them or when their time expires, and the rules are then removed. The server logs every event and samples link utilization every second. The system was evaluated with unit tests, concurrent workloads, a 25-client burst test and a traffic comparison with and without a reservation. All figures in this report come from runs on the project VM.

Objectives

1. Provide a TCP client-server interface for reservation requests (RESERVE, RELEASE, STATUS, LIST, STATS).
2. Serve many clients at the same time without allocating more than the link capacity.
3. Detect conflicts and return clear ACCEPTED or REJECTED answers with a reason.
4. Keep reservation records and free them on release or expiry.
5. Turn each accepted reservation into an SDN policy on the switches, and remove it afterwards.
6. Track link utilization and the number of accepted and rejected requests.
7. Build the network in Mininet and measure the behaviour from real runs.

Network architecture

The network has three layers: the reservation server (application), the Ryu controller (control) and two Open vSwitch switches with four hosts (data).



A client sends a TCP request to the server, the server calls the controller over REST, and the controller programs both switches over OpenFlow. Only the s1 to s2 link is limited, so that link is the resource users reserve.

Node	Address	Connected to
h1	10.0.0.1	s1 (100 Mbps access link)
h2	10.0.0.2	s1 (100 Mbps access link)
h3	10.0.0.3	s2 (100 Mbps access link)
h4	10.0.0.4	s2 (100 Mbps access link)
Reservation server	10.0.0.100, TCP port 5000	s1, from the VM's own network, so it can also reach the controller
Ryu controller	REST API on port 8080	Both switches, over OpenFlow 1.3
s1 to s2 link	10 Mbps	The reservable bottleneck

Implementation details

The code is split into small files, each with one job, so every part can be tested on its own.

File	Job
server/resource_manager.py	Holds the reservations and the 10 Mbps capacity; accepts or rejects; handles release, expiry, the event log and utilization samples
server/server.py	TCP server on port 5000; one thread per client; reads one JSON request per line and replies with one JSON line
server/sdn_policy.py	Sends HTTP requests to the controller when a reservation is accepted, released or expired
client/client.py	Command-line client: reserve, release, status, list, stats
controller/reservation_controller.py	Ryu app: learning switch plus a REST API that installs and removes meters and flow rules
topology/topo.py	Mininet network: two switches, four hosts, a 10 Mbps link between the switches
tools/	workload.py (random concurrent requests), experiment.py (the three experiments), summarize.py (numbers from the logs), make_graphs.py (graphs from the CSV files)
tests/	23 automated tests for the manager, the server and the controller messages

TCP protocol: Every message is one JSON object followed by a newline. A request looks like `{"cmd":"RESERVE","src":"10.0.0.1","dst":"10.0.0.3","bandwidth_mbps":4,"duration_s":60}`. The reply is ACCEPTED with an id, or REJECTED with a reason (INSUFFICIENT_BANDWIDTH, DUPLICATE_FLOW or POLICY_FAILED) and the bandwidth still available. TCP is used because a reservation request must arrive reliably and in order.

Concurrency and conflict detection: The server starts one thread per connected client. All reservation data sits behind one lock. The check of free bandwidth and the allocation happen inside the same lock, so two clients can never both take the last free bandwidth. A request is also rejected if the same source and destination pair already has a reservation.

Expiry and utilization: A background thread removes reservations whose time has run out and asks the controller to delete their rules. Another thread writes the reserved share of the link (percent) to a CSV file once per second. Every accept, reject, release and expiry is written to an event log.

SDN policy on each switch. The controller installs three kinds of rules.

Priority	Rule	Effect
100	One per reservation: match IPv4 source and destination	Send through a meter that drops traffic above the reserved rate, then forward
50	Best-effort limiter, present while reservations exist	Send all other IPv4 traffic through a meter set to capacity minus total reserved (never below 100 kbps)
1	Learning switch	Normal forwarding for ARP and everything else

A meter only caps a flow; it does not guarantee a minimum rate. The priority 50 limiter is what stops unreserved traffic from using the reserved share.

Integration and failure handling: The server calls the controller at `POST /policy` and `DELETE /policy/<id>` (port 8080). If the controller cannot install the rule, the reservation is refused with POLICY_FAILED and the bandwidth is given back, so the server and the switches never disagree.

Test cases and simulation screenshots

No.	Test case	Expected	Observed	Result	Evidence
T1	Network reachability	All four hosts reach each other	pingall: 0% dropped (12/12 received)	Pass	Figure 1
T2	Controller sees both switches	Switches 1 and 2 connect to Ryu	Log shows “switch connected” for dpid 1 and dpid 2	Pass	Figure 2
T3	Valid request	Accepted, and the rule is installed	4 Mbps h1 to h3 for 20 s: ACCEPTED, id 3; server log shows “install id=3 … OK”	Pass	Figures 3, 4
T4	Over-capacity request	Rejected, with the bandwidth still available	7 Mbps requested with 4 Mbps in use: REJECTED, INSUFFICIENT_BANDWIDTH, available 6.0	Pass	Figure 3
T5	Exact fit	Accepted when the request equals the free bandwidth	6 Mbps requested: ACCEPTED, id 4 (total 10 of 10 Mbps)	Pass	Figure 3
T6	List reservations	Shows the active reservation and its remaining time	LIST returns id 4 with remaining_s 8.5	Pass	Figure 3
T7	Expiry	Bandwidth freed and rules removed after the duration	Server log shows “remove id=3” and “remove id=4” after their 20 s	Pass	Figure 4
T8	Duplicate flow	Second reservation for the same source and destination is rejected	DUPLICATE_FLOW rejections in the workload runs (3 and 6 requests); unit test passes	Pass	Figure 5, results/
T9	Release	Bandwidth freed at once	Unit test test_release_frees_bandwidth passes; the controller receives a DELETE	Pass	Figure 5
T10	Controller unavailable	Reservation refused with POLICY_FAILED; no bandwidth kept	Unit tests with a fake controller pass (controller down and controller error). Not repeated live with Ryu stopped	Pass (unit tested)	Figure 5
T11	Concurrent clients	No overbooking	25 simultaneous requests: at most 9.7 of 10 Mbps reserved on the server and both switches; no meters left after expiry	Pass	results/
T12	Load levels	Accepted and rejected counts are recorded	Overload run 11 of 30 accepted; heavy run 6 of 30 accepted; peak utilization 96% and 99%	Pass	results/
T13	Baseline vs reservation	Reserved stream gets more with a reservation	1.90 Mbps without, 3.57 Mbps with (3 runs each)	Pass	results/

Figure 1: Mininet topology and reachability

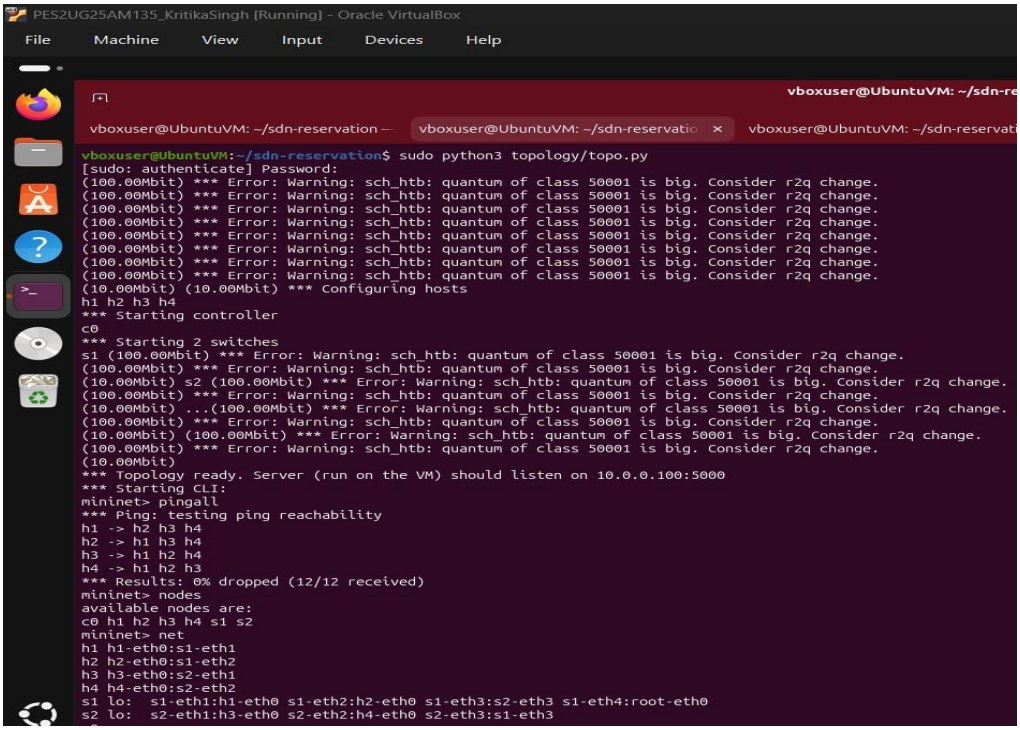


Figure 2: Ryu controller

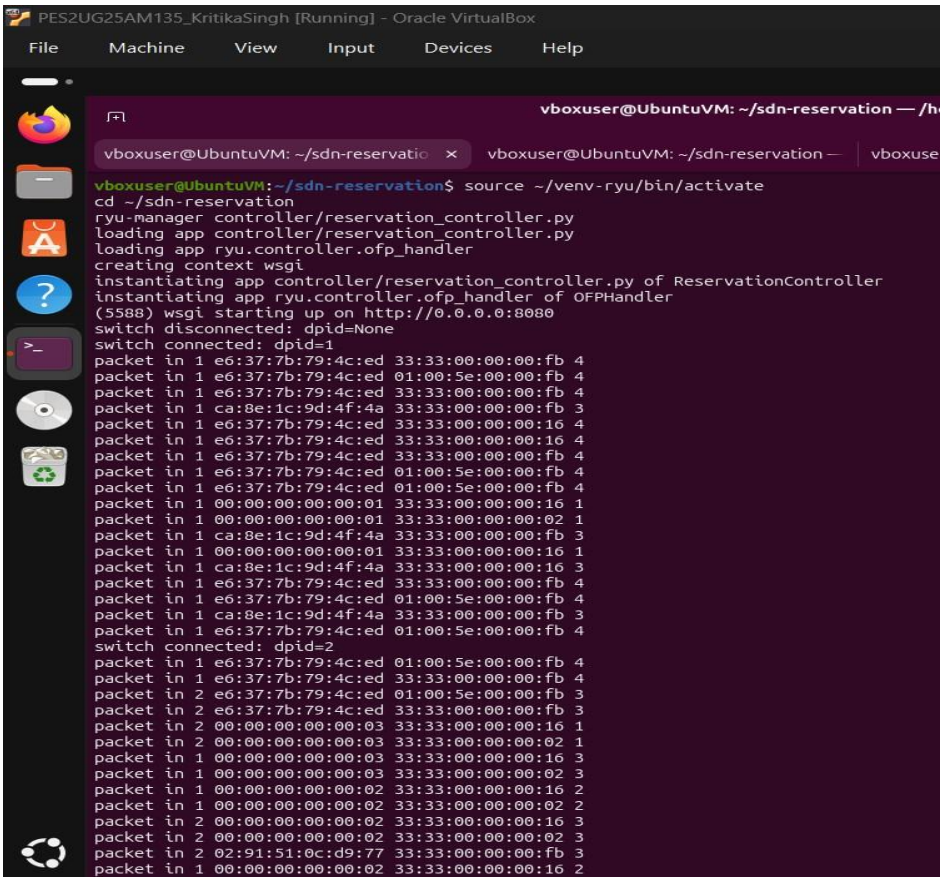


Figure 3: Client requests-- An accepted 4 Mbps request, a rejected 7 Mbps request with 6.0 Mbps still available, an accepted 6 Mbps request, and the list of active reservations.

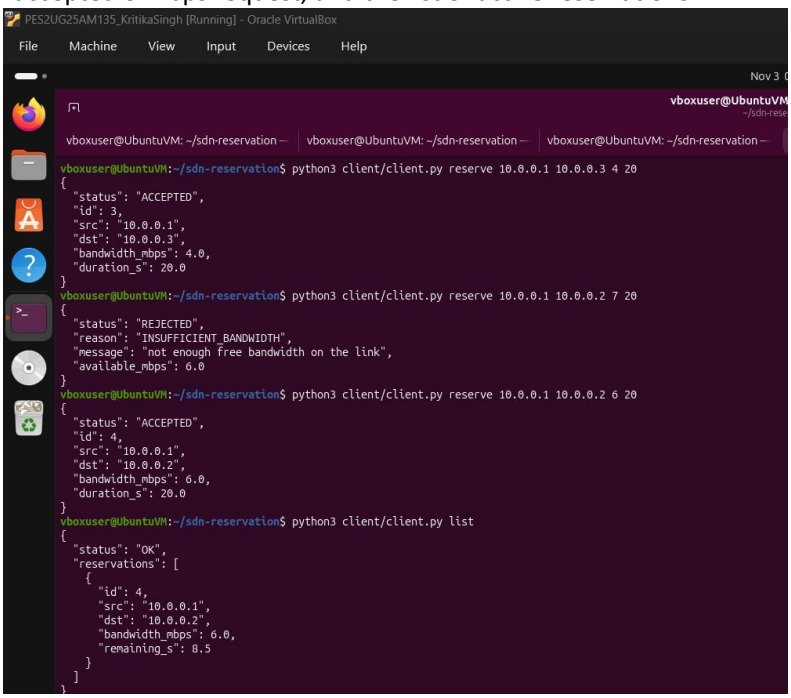


Figure 4: Server log

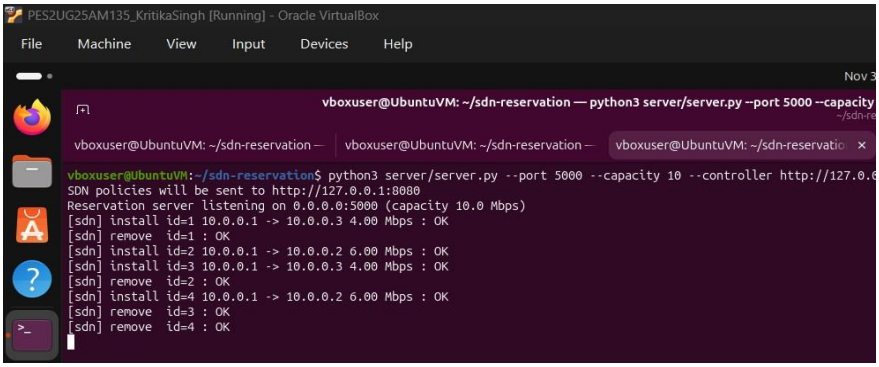
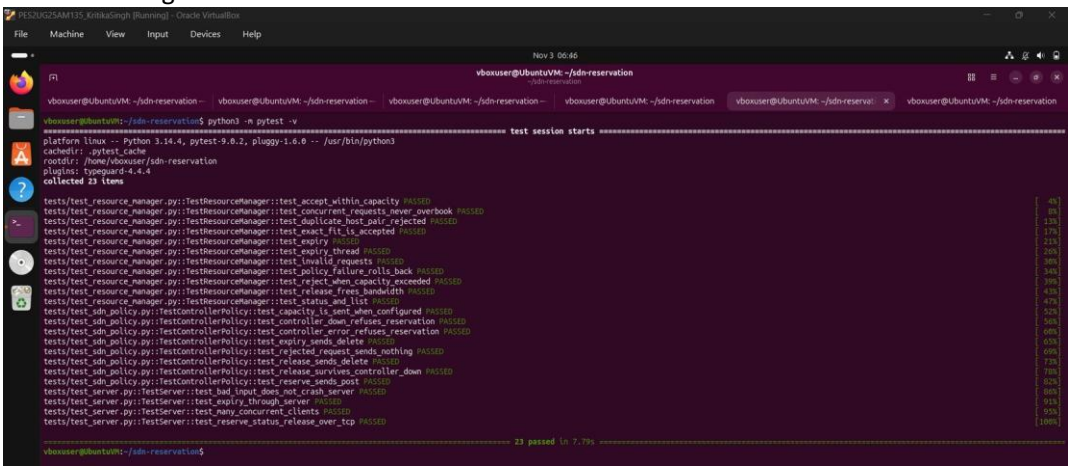


Figure 5: Automated tests-- All 23 tests pass, covering the resource manager, the server over TCP, and the controller messages.



Results and conclusion

A reservation protected the reserved stream in every run, and the server never allocated more than the 10 Mbps link. Every number below was measured on the project VM(Ubuntu), the raw CSV files and graphs are in the `results/` folder of the repository.

Experiment	Measured result
Automated tests	23 of 23 passed (7.79 s)
Baseline vs reservation (UDP, 3 runs each, four greedy streams competing)	Reserved stream received 1.90 Mbps without a reservation and 3.57 Mbps with a 4 Mbps reservation
Same experiment, greedy streams	Total 7.73 Mbps without a reservation and 5.89 Mbps with one, so they were held back to the leftover capacity
Burst test: 25 clients ask at the same instant	At most 9.7 of 10 Mbps reserved, the same on the server and on both switches; the best-effort limiter was set to 300 kbps; no overbooking; no meters left after expiry
Overload workload (30 requests)	11 accepted (36.7%), 19 rejected: 16 INSUFFICIENT_BANDWIDTH and 3 DUPLICATE_FLOW; server reply time mean 6.3 ms, median 1.8 ms, maximum 43.9 ms; peak utilization 96%
Heavy workload (30 requests, one every 0.2 s)	6 accepted (20%), 24 rejected: 18 INSUFFICIENT_BANDWIDTH and 6 DUPLICATE_FLOW; reply time mean 3.0 ms; peak utilization 99%

Comments.

- Without a reservation, the stream that sends 3.4 Mbps got only about half of it, because four greedy streams shared the 10 Mbps link. With a reservation it got its 3.57 Mbps.
- Rejections are correct, not failures: once the link is nearly full, further requests must be refused. The heavy workload shows this: peak utilization 99%.
- A meter alone caps a flow but does not protect it. The reserved stream only improved once the best-effort limiter was added to cap all other traffic.

Limitations.

- Protection of the reserved flow depends on the meter and the best-effort limiter working together.
- Everything ran on one virtual machine, not on real network hardware.

Conclusion. The project meets all eight mandatory features: a TCP reservation protocol, concurrent clients, resource information, conflict detection, SDN policy installation, reservation expiry, utilization tracking, and an accepted versus rejected evaluation. An accepted request becomes a verified rule on the switches, and the rule disappears on release or expiry. The tests and experiments show that the system never overbooks the link and that a reservation measurably improves the reserved stream under competing traffic.

Appendix

A. Environment

- Ubuntu virtual machine in Oracle VirtualBox
- Mininet and Open vSwitch (topology and switches), OpenFlow 1.3
- System Python 3.14 for the server, client, topology, tools and tests
- Python 3.9 virtual environment (`~/venv-ryu`) with Ryu 4.34 for the controller only
- Ports: reservation server 5000, controller REST API 8080; server address inside the network 10.0.0.100

B. How to run (order matters)

Running `sudo mn -c` also stops the controller, so always clean up first and start the controller second.

```
# Terminal 1: clean up, then start the controller
sudo mn -c
source ~/venv-ryu/bin/activate
cd ~/sdn-reservation
ryu-manager controller/reservation_controller.py

# Terminal 2: start the network
sudo python3 topology/topo.py

# Terminal 3: start the reservation server
python3 server/server.py --port 5000 --capacity 10 \
  --controller http://127.0.0.1:8080 \
  --log results/events.csv --samples results/utilization.csv

# Terminal 4: send requests
python3 client/client.py reserve 10.0.0.1 10.0.0.3 4 20
python3 client/client.py list
# Tests
python3 -m pytest -v
```

C. Repository structure-

Folder	Contents
server/	resource_manager.py, server.py, sdn_policy.py
client/	client.py
controller/	reservation_controller.py
topology/	topo.py
tools/	workload.py, experiment.py, summarize.py, make_graphs.py
tests/	test_resource_manager.py, test_server.py, test_sdn_policy.py
results/	CSV files, raw outputs and graphs from the experiments

D. Further evidence- The experiment graphs (baseline versus reservation, throughput over time, accepted and rejected requests by load, utilization over time) and the CSV files are in `results folder` and have been uploaded to the GitHub repository.